k-Nearest Neighbors (kNN) classifier using the Iris dataset

Python codes for k-Nearest Neighbors (kNN) classifier using the Iris dataset and then testing it with an unknown sample (Jupyter Notebook)

1. Import Libraries

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report
```

2. Load the Iris Dataset

In [2]:
```
iris = load_iris()

X = iris.data       # Features
y = iris.target     # Labels

print("Feature names:", iris.feature_names)
print("Target names:", iris.target_names)
```

```
Feature names: ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal
width (cm)']
Target names: ['setosa' 'versicolor' 'virginica']
```

3. Split the Dataset

In [3]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("Training samples:", X_train.shape[0])
print("Testing samples:", X_test.shape[0])
```

```
Training samples: 120
Testing samples: 30
```

4. Train kNN Classifier

In [4]:
```
k = 3  # number of neighbors
knn = KNeighborsClassifier(n_neighbors=k)

knn.fit(X_train, y_train)
```

Out[4]:
```
KNeighborsClassifier
KNeighborsClassifier(n_neighbors=3)
```

5. Evaluate the Model

In [5]:
```
y_pred = knn.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

```
Accuracy: 1.0

Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        10
           1       1.00      1.00      1.00         9
           2       1.00      1.00      1.00        11

    accuracy                           1.00        30
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30
```

6. Test with Unknown Data

In [6]:
```
# Example unknown data (sepal length, sepal width, petal length, petal width)
unknown_sample = np.array([[5.1, 3.5, 1.4, 0.2]])

prediction = knn.predict(unknown_sample)
predicted_class = iris.target_names[prediction[0]]

print("Predicted class for unknown sample:", predicted_class)
```

```
Predicted class for unknown sample: setosa
```

7. Try Multiple Unknown Samples

In [7]:
```
unknown_samples = np.array([
    [6.7, 3.1, 4.7, 1.5],  # likely versicolor
    [7.2, 3.6, 6.1, 2.5],  # likely virginica
    [5.0, 3.4, 1.5, 0.2]   # likely setosa
])

predictions = knn.predict(unknown_samples)

for i, pred in enumerate(predictions):
    print(f"Sample {i+1} predicted as:", iris.target_names[pred])
```

```
Sample 1 predicted as: versicolor
Sample 2 predicted as: virginica
Sample 3 predicted as: setosa
```

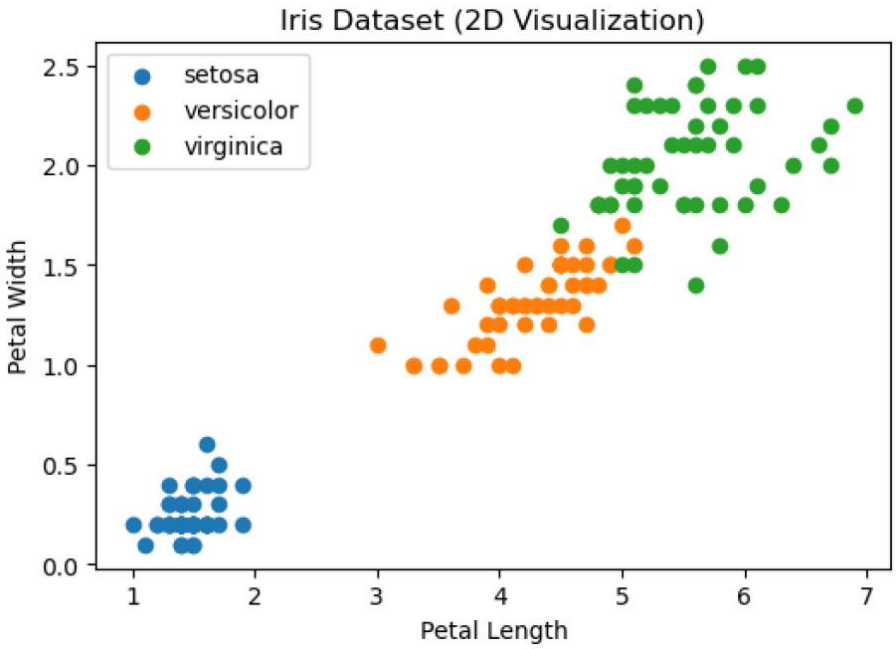
Visualization of the data and the kNN decision boundaries using only two features (for plotting in 2D): petal length and petal width

8. Visualize the Dataset (2D Projection)

In [8]:
```
# Use only two features: petal length (2) and petal width (3)
X_vis = X[:, 2:4]
y_vis = y

plt.figure(figsize=(6, 4))
for i, target_name in enumerate(iris.target_names):
    plt.scatter(
        X_vis[y_vis == i, 0],
        X_vis[y_vis == i, 1],
        label=target_name
    )

plt.xlabel("Petal Length")
plt.ylabel("Petal Width")
plt.legend()
plt.title("Iris Dataset (2D Visualization)")
plt.show()
```



9. Train kNN on 2D Data

In [9]:
```
X_train_vis, X_test_vis, y_train_vis, y_test_vis = train_test_split(
    X_vis, y_vis, test_size=0.2, random_state=42
)

knn_vis = KNeighborsClassifier(n_neighbors=3)
knn_vis.fit(X_train_vis, y_train_vis)
```

Out[9]:
```
KNeighborsClassifier
KNeighborsClassifier(n_neighbors=3)
```

10. Plot Decision Boundary

In [10]:
```
# Create mesh grid
h = 0.02  # step size
x_min, x_max = X_vis[:, 0].min() - 1, X_vis[:, 0].max() + 1
y_min, y_max = X_vis[:, 1].min() - 1, X_vis[:, 1].max() + 1

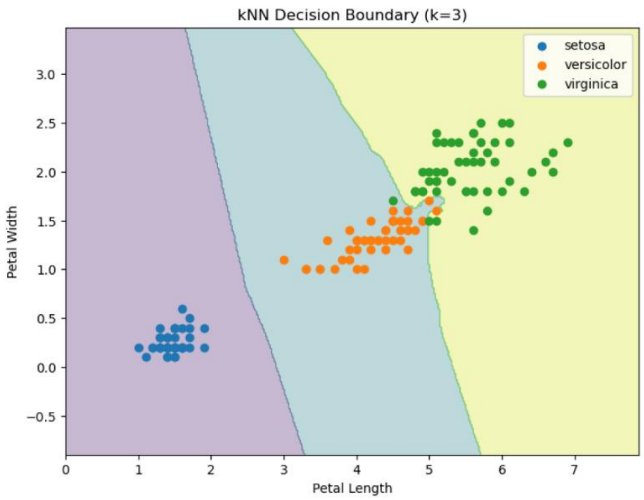
xx, yy = np.meshgrid(
    np.arange(x_min, x_max, h),
    np.arange(y_min, y_max, h)
)

# Predict over grid
Z = knn_vis.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

# Plot decision boundary
plt.figure(figsize=(8, 6))
plt.contourf(xx, yy, Z, alpha=0.3)

# Plot actual points
for i, target_name in enumerate(iris.target_names):
    plt.scatter(
        X_vis[y_vis == i, 0],
        X_vis[y_vis == i, 1],
        label=target_name
    )

plt.xlabel("Petal Length")
plt.ylabel("Petal Width")
plt.legend()
plt.title("kNN Decision Boundary (k=3)")
plt.show()
```



11. Visualize Unknown Sample Classification

In [11]:
```
# Unknown sample (same 2 features: petal length, petal width)
unknown = np.array([[1.5, 0.3]])

pred = knn_vis.predict(unknown)
label = iris.target_names[pred[0]]

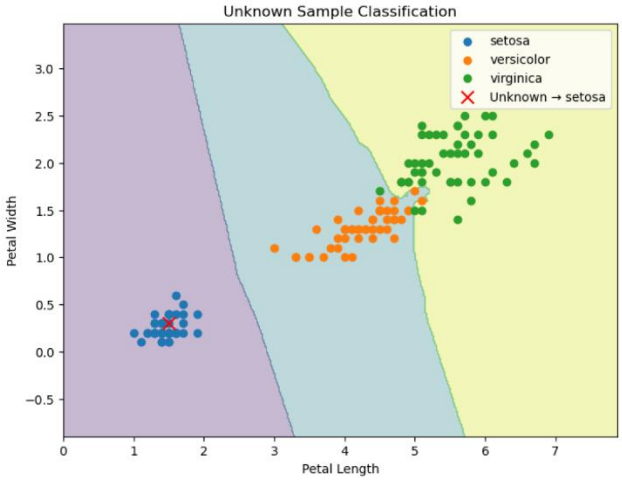
plt.figure(figsize=(8, 6))
plt.contourf(xx, yy, Z, alpha=0.3)

# Plot dataset
for i, target_name in enumerate(iris.target_names):
    plt.scatter(
        X_vis[y_vis == i, 0],
        X_vis[y_vis == i, 1],
        label=target_name
    )

# Plot unknown point
plt.scatter(unknown[:, 0], unknown[:, 1],
            color='red', marker='x', s=100,
            label=f"Unknown → {label}")

plt.xlabel("Petal Length")
plt.ylabel("Petal Width")
plt.legend()
plt.title("Unknown Sample Classification")
plt.show()

print("Predicted class:", label)
```




```
Predicted class: setosa
```